



Lecture[05]

INTERNAL SORTING



Agenda

- Sorting Terminology and Notation
 - Key and record 关键字和记录
 - Comparisons
 - Swaps
- Three $O(n^2)$ Sorting Algorithms
 - Insertion Sort
 - Selection Sort
 - Bubble Sort
- Shell Sort
- QuickSort
 - Recursive function 递归函数
 - Partition function 分区函数
 - Complexity of QuickSort



Sorting

Each record contains a field called the key.

- Linear order: comparison.

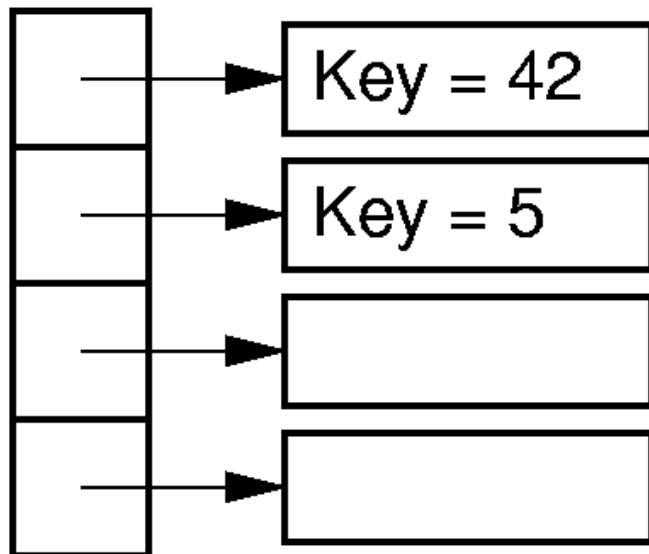
Measures of cost:

- Comparisons

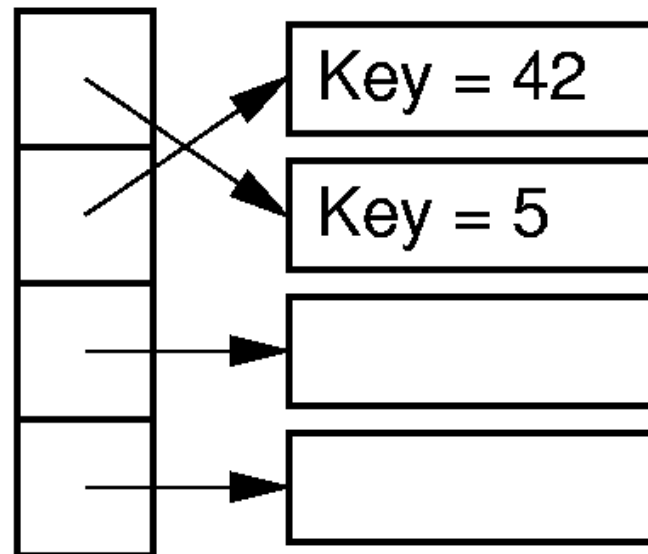
- Swaps



Pointer Swapping



(a)



(b)



swap function

```
void swap(int *A, int i, int j)
{
    int temp;
    temp=A[i];
    A[i]=A[j];
    A[j]=temp;
}
```



Insertion Sort (1)

	i=1	2	3	4	5	6	7
<u>42</u>	<u>20</u>	<u>17</u>	13	13	13	13	13
20	42	20	17	17	14	14	14
17	17	42	20	20	17	17	15
13	13	13	42	28	20	20	17
28	28	28	28	42	28	23	20
14	14	14	14	14	42	28	23
23	23	23	23	23	23	42	28
15	15	15	15	15	15	15	<u>42</u>



Insertion Sort (2)

```
void inssort(int* A, int N)
{
    int i, j, temp;
    for (i = 1; i < N; i++)
    {
        j = i - 1;
        temp = A[i];
        while (j >= 0 && A[j] > temp)
        {
            swap(A, j, j+1);
            j--;
        }
    }
}
```

Comparison	Swap
Best Case: $O(n)$	Best Case: 0
Worst Case: $O(n^2)$	Worst Case: $O(n^2)$
Average Case: $O(n^2)$	Average Case: $O(n^2)$

Best Case (already ordered in the same order as sort): $O(n)$

Worst Case (already ordered in the inversed order as sort): $O(n^2)$

Average Case: $O(n^2)$



Selection Sort (1)

	i=0	1	2	3	4	5	6
42	<u>13</u>	13	13	13	13	13	13
20	20	<u>14</u>	14	14	14	14	14
17	17	17	<u>15</u>	15	15	15	15
13	42	42	42	<u>17</u>	17	17	17
28	28	28	28	28	<u>20</u>	20	20
14	14	20	20	20	28	<u>23</u>	23
23	23	23	23	23	23	28	<u>28</u>
15	15	15	17	42	42	42	42



Selection Sort (2)

```
void selsort(int* A, int N)
{
    int i, j, index;
    for ( i = 0; i < N - 1; i++)
    {
        index = i;
        for ( j = i + 1; j < N; j++)
            if (A[j] < A[index])
                index = j;
        if (index != i)
            swap(A, index, i);
    }
}
```

Best Case: $O(n^2)$

Worst Case: $O(n^2)$

Average Case: $O(n^2)$

Comparison	Swap
Best Case: $O(n^2)$	Best Case: 0
Worst Case: $O(n^2)$	Worst Case: $O(n)$
Average Case: $O(n^2)$	Average Case: $O(n)$



Bubble Sort (1)

	i=0	1	2	3	4	5	6
42	13	13	13	13	13	13	13
20	42	14	14	14	14	14	14
17	20	42	15	15	15	15	15
13	17	20	42	17	17	17	17
28	14	17	20	42	20	20	20
14	28	15	17	20	42	23	23
23	15	28	23	23	23	42	28
15	23	23	28	28	28	28	42



Bubble Sort (2)

```
void bubsort(int* A, int N)
{
    int i, j;
    for (i = 0; i < N-1; i++)
        for (j = 0; j < N-i-1; j++)
            if (A[j] > A[j+1])
                swap(A, j, j+1);
}
```

Comparison	Swap
Best Case: $O(n^2)$	Best Case: 0
Worst Case: $O(n^2)$	Worst Case: $O(n^2)$
Average Case: $O(n^2)$	Average Case: $O(n^2)$

Best Case: $O(n^2)$
Worst Case: $O(n^2)$
Average Case: $O(n^2)$



Summary

	Insertion	Bubble	Selection
Comparisons:			
Best Case	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Worst Case	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Average Case	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Swaps			
Best Case	0	0	0
Worst Case	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$
Average Case	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$



Exchange Sorting

All of the sorts so far rely on exchanges of adjacent records.

What is the average number of exchanges required?

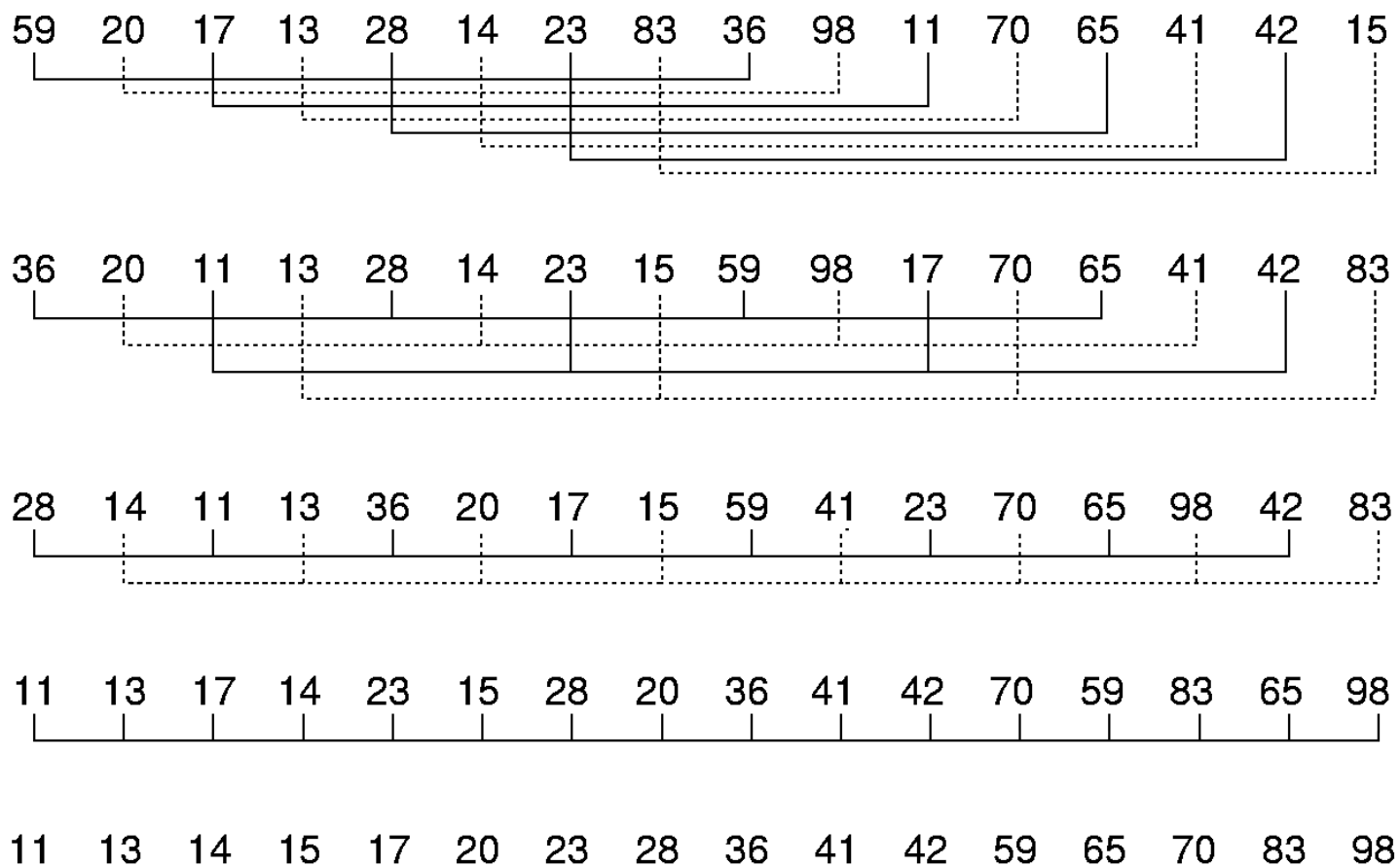
- There are $n!$ permutations
- Consider permutation X and its reverse X' together, every pair requires $n(n-1)/2$ exchanges.

Is there a faster algorithm?



Shellsort (1)

Shell's Sort is a type of insertion sort, also known as **Diminishing Increment Sort**, and is a more efficient improved version of the direct insertion sort algorithm.





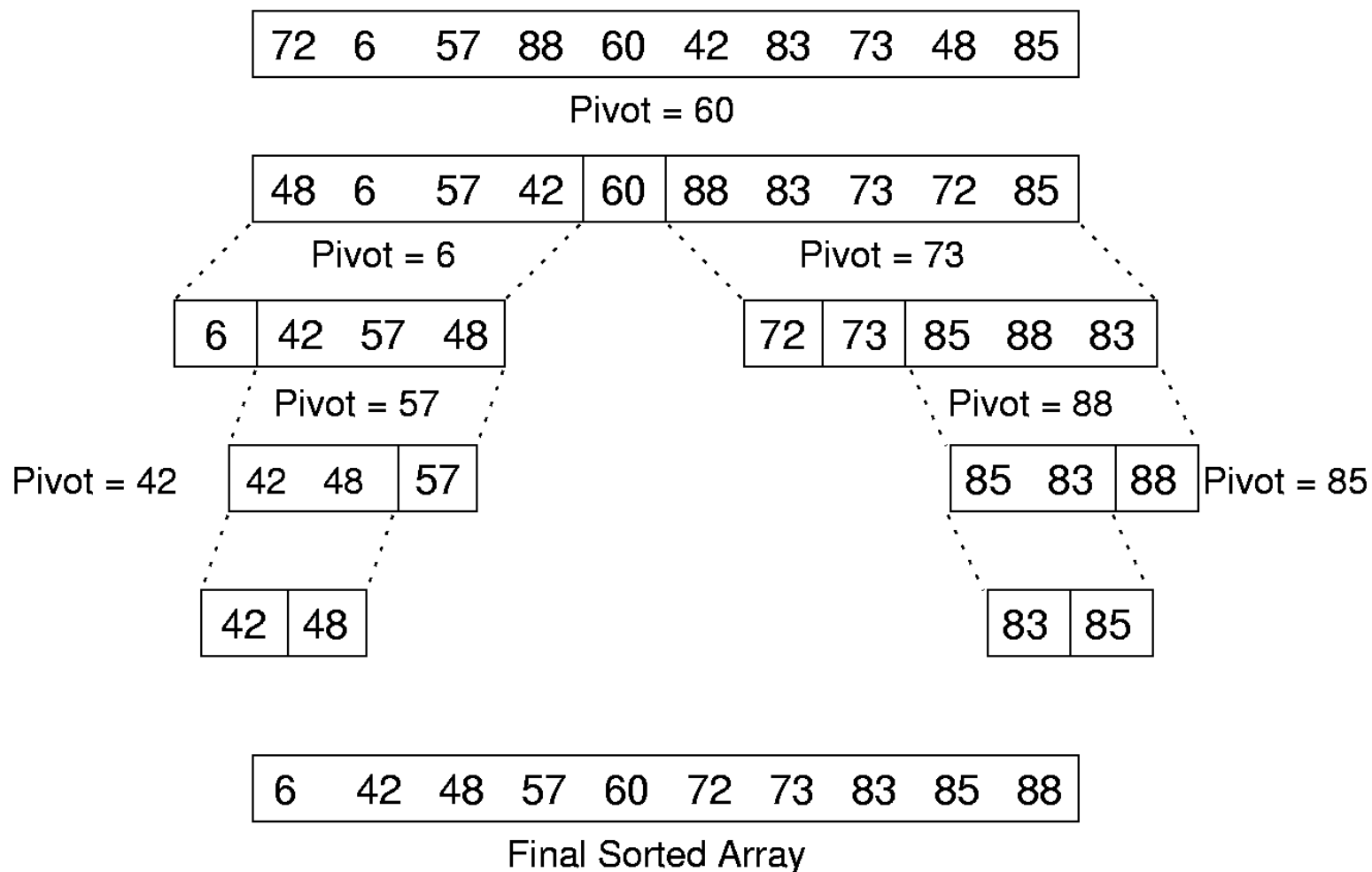
Shellsort (2)

```
void shesort(int* A, int N)
{
    int i, j, Increment, Tmp;
    for( Increment = N / 2; Increment > 0; Increment /= 2 )
        for( i = Increment; i < N; i++ )
        {
            Tmp = A[ i ];
            for( j = i; j >= Increment; j -= Increment )
                if( Tmp < A[ j - Increment ] )
                    A[ j ] = A[ j - Increment ];
            else
                break;
            A[ j ] = Tmp;
        }
}
```

The direct insertion sort wastes a lot of time searching and moving elements. It is find that after the first three rounds of Shell's Sort, the sequence is already very ordered. In other words, it is very close to where it should be. Therefore, when performing direct insertion sorting, it saves a lot of time!



Quicksort (1)



Best case?
Worst case?



Quicksort (2)

```
void quisort(int* A, int start, int end)
{
    int i;
    if( end > start )
    {
        i = partition( A, start, end );
        quisort( A, start, i - 1 );
        quisort( A, i + 1, end );
    }
}
```



Partition Example

Init	72	6	57	88	60	42	83	73	48	85
	l									r
l/r move?	l								r	
swap	48	6	57	88	60	42	83	73	72	85
	l								r	
l/r move?				l		r				
swap	48	6	57	42	60	88	83	73	72	85
				l		r				
					l,r					
break out										

```

while( 1 )
{
    while( A[ l ] < pivo ){ l++; }
    while( A[ r ] > pivo ){ r--; }
    if ( l < r )
        swap(A, l, r);
    else
        break;
}

```



Quicksort Partition (different elements)

```
int partition(int* A, int left, int right)
{
    int i, j;
    int piv = A[ (left+right)/2 ];
    i = left;    j = right;

    while( 1 )
    {
        while( A[ i ] < piv ) { i++; }
        while( A[ j ] > piv ) { j--; }
        if ( i < j )
            swap(A, i, j);
        else
            break;
    }
    return i;
}
```

The cost for partition is $\Theta(n)$.



Partition Example 2

Init	72	6	57	40	42	60	83	73	48	85
	l									r
l/r move?	l				r					
swap	42	6	57	40	72	60	83	73	48	85
	l				r					
l/r move?	l			r						
swap	40	6	57	42	72	60	83	73	48	85
	l			r						
l/r move?			l	r						
swap	40	6	42	57	72	60	83	73	48	85
			l	r						
break out			l,r							

```

while( 1 )
{
    while( A[ l ] < pivo ){ l++; }
    while( A[ r ] > pivo ){ r--; }
    if ( l < r )
        swap(A, l, r);
    else
        break;
}

```



Partition Example 3

Put pivot to the end of the array

Init	72	6	57	40	42	60	83	73	48	85
swap(A,pIn,right)	72	6	57	40	85	60	83	73	48	42
	l								r	
l/r move?	l			r						
swap	40	6	57	72	85	60	83	73	48	42
	l			r						
breakout		r	l							
swap(A,l,right)	40	6	42	72	85	60	83	73	48	57

```
int pIn=(left+right)/2;
swap(A,pIn,right);
l = left; r = right-1;
while( 1 )
{
    while( A[ l ] < pivo ){ l++; }
    while( A[ r ] > pivo ){ r--; }
    if ( l < r )
        swap(A, l, r);
    else
        break;
}
swap(A,l,right);
```



Partition Example 4

Select mid value of three positions

Init	72	6	57	40	42	60	83	73	48	85
arrange	42	6	57	40	72	60	83	73	48	85
	left				mid					right
swap(A,pIn,right-1)	42	6	57	40	48	60	83	73	72	85
	l							r		
breakout						r	l			
swap(A,l,right-1)	42	6	57	40	48	60	72	73	83	85

Find order of elements on left, right and $(\text{left} + \text{right}) / 2$;

Arrange them in order;

swap(A,pIn,right-1);

$l = \text{left}; r = \text{right} - 2$;

while(1)

{

while($A[l] < \text{pivo}$){ $l++$; }

while($A[r] > \text{pivo}$){ $r--$; }

if ($l < r$)

swap(A, l, r);

else

break;

}

swap(A,l,right-1);



Cost of Quicksort

Best case: Always partition in half.

$O(n \log(n))$

Worst case: Bad partition.

$O(n^2)$

Average case:

$$T(n) = cn + \frac{1}{n} \sum_{k=0}^{n-1} (T(k) + T(n-k))$$

$O(n \log(n))$

Optimizations for Quicksort:

- Better Pivot
- Better algorithm for small sublists
- Eliminate recursion



Summary

- Complexity of $O(n^2)$ sorting algorithm
 - Best/average/worst case
- Complexity of QuickSort
 - Best/average/worst case
- Implementation of insertion/selection/bubble/quicksort with an array